

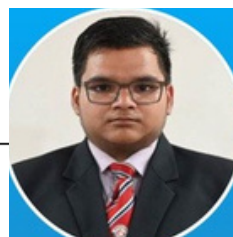
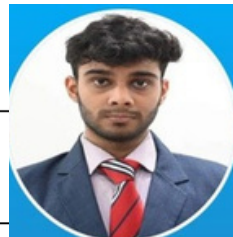
PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Cybercrime Evidence & Attack Relationship Analyzer

Student / Team Information

Team Name: Tech Hunter Team #	
Team member 1 (Team Lead): Bansal Gaurav: ID: 2510360068 E-mail: 2510360068@geu.ac.in	
Team member 2: Jain Ketan: ID: 25101281877 E-mail: 25101281877@geu.ac.in	
Team member 3: Garg Shubhangi: ID: 2510012138 E-mail: 2510012138@geu.ac.in	
Team member 4: Jain Saanvi: ID: 2510011590 E-mail: 2510011590@geu.ac.in	

PROPOSAL DESCRIPTION

Motivation

Cybercrime investigations involve digital artifacts such as IP addresses, device identifiers, user accounts, domains, files, timestamps, and collected evidence. When these artifacts are stored separately, it can be difficult to search evidence efficiently and understand how different artifacts are connected. The Cybercrime Evidence & Attack Relationship Analyzer aims to provide an academic C++ system for organizing digital evidence and analysing relationships between related artifacts. The project demonstrates how Hash Tables, Linked Lists, Stacks/Queues, Searching, Sorting, and Graphs can be combined to manage evidence and reveal suspicious attack relationships in a transparent and understandable way.

State of the Art / Current Solution

Modern digital-forensics and security platforms collect and correlate information from multiple sources using evidence repositories, logs, timelines, indicators, and relationship graphs. Professional platforms may also use automated correlation and advanced analytics. This project intentionally uses a simpler rule-based approach for academic demonstration. It focuses on fundamental Data Structures and Algorithms rather than Artificial Intelligence or Machine Learning, so each processing step remains understandable and implementable in C++.

Project Goals and Milestones

Phase 1 — Planning & Requirement Analysis: Define evidence fields, case workflow, relationship types, scope, and required C++ data structures.

Phase 2 — System Design & Development: Design the architecture, evidence model, relationship graph, search/sort modules, rule-based analysis, and file handling.

Phase 3 — Testing & Final Implementation: Test sample cases, verify relationships, handle invalid data, generate reports, and prepare final documentation/demo.

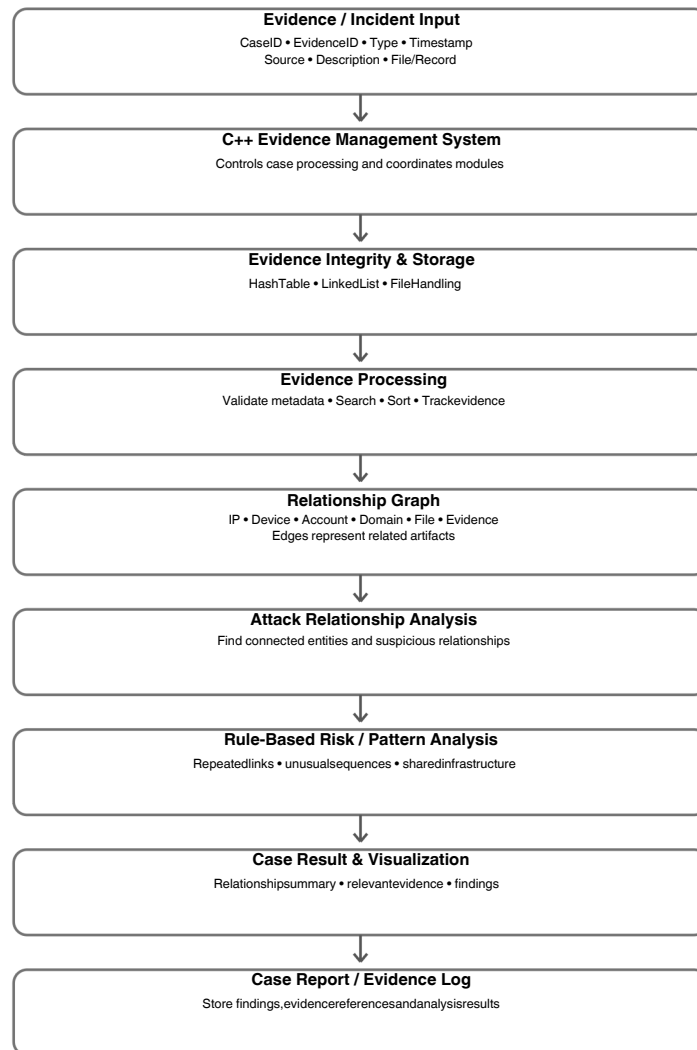
Project Approach

The Cybercrime Evidence & Attack Relationship Analyzer will be developed as a terminal-based C++ application using Data Structures and Algorithms. Structures and classes will represent cases and evidence records containing Case ID, Evidence ID, type, source, timestamp, description, and related artifact identifiers. Arrays or vectors can store collections of records, while a linked list can maintain dynamically changing evidence entries. A queue can process incoming evidence records in order, and a stack can maintain recent analysis history. A Hash Table can provide fast lookup of evidence or artifact identifiers. Searching and sorting will locate and organize evidence by timestamp, type, priority, or risk. A Graph will model relationships among IP addresses, devices, accounts, domains, files, and evidence records. Graph traversal can discover connected artifacts and possible attack relationships. Rule-based checks will analyse repeated connections, shared infrastructure, suspicious sequences, and other predefined conditions. File handling will store case records, evidence information, relationships, and findings. Finally, the system will produce a relationship summary and case/evidence log.

Planned Core Modules

Module	Purpose	DSA / C++ Concept
Case & Evidence Input	Enter and validate records	Structures, Classes
Evidence Storage	Maintain evidence	Vector, Linked List
Fast Lookup	Find artifacts quickly	Hash Table
Processing	Track incoming/recent records	Queue / Stack
Search & Sort	Locate and organize evidence	Searching, Sorting
Relationship Analyzer	Model connected artifacts	Graph + Traversal
Rule Engine	Flag suspicious patterns	Conditional rules
Reports & Logs	Save findings	File Handling

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The project will deliver a functional terminal-based C++ application for managing cybercrime evidence and analysing relationships between digital artifacts. Deliverables will include Case and Evidence data structures, Hash Table lookup, Linked List/Vector storage, Stack/Queue processing, Searching and Sorting modules, Relationship Graph and traversal, predefined rule-based checks, suspicious relationship logging, sample datasets, file handling, test cases, documentation, and final demonstration.

Assumptions

The project assumes that each evidence record contains a valid Case ID, Evidence ID, timestamp, source, and basic artifact information. Sample data is assumed to be sufficient to demonstrate relationships between digital artifacts. Relationship rules are predefined for academic testing and do not represent professional forensic conclusions. The system is intended for educational demonstration and is not a replacement for certified digital-forensics software or an investigation platform.

References

1. Thomas H. Cormen et al., Introduction to Algorithms, MIT Press.
2. Bjarne Stroustrup, The C++ Programming Language, Addison-Wesley.
3. cppreference, C++ Standard Library and Container Documentation.
4. GeeksforGeeks, Data Structures and Algorithms in C++.
5. Course material and lecture notes on Data Structures and Algorithms.